HR Assistant RAG Pipeline

Production Architecture, Implementation Guide & Complete Technical Interview Playbook

1. Executive Summary & What is RAG?

Retrieval-Augmented Generation (RAG) is an enterprise architecture that equips Large Language Models (LLMs) with private, dynamic, and authoritative company knowledge without fine-tuning weights. For the **HR Assistant**, it guarantees accurate responses on sensitive topics such as leave policies, probation rules, benefits, and resignation guidelines while eliminating hallucinations.

RAG vs Fine-Tuning: Fine-tuning bakes static data into weights (expensive, slow, risk of catastrophic forgetting, no granular ACLs). RAG keeps enterprise data external, allowing instant policy updates, role-based document access, and verifiable citations.

2. HR Assistant End-to-End Pipeline Architecture

Ingestion Phase (Offline / Document Sync)	Query & Generation Phase (Online / Low-Latency)
• **Document Loaders:** PDF/DOCX policy extraction (pdf-parse, mammoth). • **Chunking Engine:** 400-600 token windows with 50-token semantic overlap. • **Embedding Model:** Vectorized using OpenAI text-embedding-3-small (1536 dims) or Google text-embedding-004. • **Vector Store:** PostgreSQL + **pgvector** with HNSW / IVFFlat indexes. • **Access Tagging:** Chunks tagged with metadata (role, department, doc_version).	• **User Query:** Employee asks question via Webhook / Chatbot. • **Query Vectorization:** Question converted to matching 1536-d vector. • **Hybrid Retrieval:** Cosine similarity search (<=>) + BM25 keyword matching. • **Reranking:** Top-10 candidates scored via Cross-Encoder down to Top-3. • **Contextual Prompting:** Strictly grounded prompt fed to LLM (GPT-4o/Gemini). • **Response & Citation:** Grounded answer returned with document sources.

3. Recommended Tech Stacks

Component	Production Stack (Node/Postgres)	Python / AI Native Stack
Vector Database	PostgreSQL + **pgvector** (HNSW index)	Chroma / Qdrant / Pinecone
Embedding Model	OpenAI text-embedding-3-small (1536-dim)	bge-large-en-v1.5 / text-embedding-004
LLM Engine	OpenAI GPT-4o / Claude 3.5 Sonnet	Gemini 1.5 Pro / Llama 3 (Ollama/vLLM)
Backend Framework	Node.js (Express / Fastify)	Python FastAPI
Orchestration / RAG	LangChain.js or Custom Modular Services	LangChain / LlamaIndex
Parser / Extractor	pdf-parse, mammoth (DOCX)	Unstructured / PyPDF / Docling
Reranker	Cohere Rerank API	sentence-transformers CrossEncoder

4. Concrete Implementation Walkthrough

```
// 1. pgvector Schema Definition
CREATE EXTENSION IF NOT EXISTS vector;
CREATE TABLE hr_knowledge_base (
id SERIAL PRIMARY KEY,
doc_name VARCHAR(255) NOT NULL,
chunk_content TEXT NOT NULL,
embedding vector(1536) NOT NULL,
metadata JSONB,
created_at TIMESTAMP DEFAULT NOW()
);
CREATE INDEX ON hr_knowledge_base USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction =
64);
```

```
// 2. Similarity Search Query (pgvector Cosine Distance <=>)
SELECT doc_name, chunk_content,
1 - (embedding <=> $1::vector) AS similarity_score
FROM hr_knowledge_base
WHERE 1 - (embedding <=> $1::vector) > 0.72
ORDER BY embedding <=> $1::vector ASC
LIMIT 5;
```

5. Complete Technical Interview Questions & Answers

Curated categorized questions covering Fundamentals, Production Engineering, and Advanced RAG Optimizations.

Q1: Why choose RAG over fine-tuning for an internal HR assistant?

Answer: 1) **Freshness:** HR policies change frequently; updating RAG only requires re-indexing the revised document in seconds, whereas fine-tuning takes hours/days and significant compute. 2) **Hallucination reduction:** RAG forces the LLM to ground its answers strictly in retrieved chunks. 3) **Access Control (ACLs):** RAG allows metadata filtering so junior staff cannot retrieve executive salary bands or confidential disciplinary records. 4) **Verifiability:** RAG provides exact citations and document titles with page numbers for transparency.

Q2: How do you choose the ideal chunk size and overlap for HR documents?

Answer: Typical policy documents perform best with **400 to 600 tokens** with **10-15% (50 tokens) overlap**. Smaller chunks (~100 tokens) lose semantic context (e.g., qualifying conditions split from rule definitions). Larger chunks (>1000 tokens) dilute embedding representations and cause irrelevant background text to waste LLM context window tokens. Recursive character splitting (splitting by double newline, newline, period, space) preserves structural paragraph coherence.

Q3: Explain the difference between HNSW and IVFFlat indexes in pgvector.

Answer: IVFFlat (Inverted File Flat) clusters vectors into Voronoi cells. It has fast build times and low memory footprint, but query recall drops unless higher 'probes' are searched, and it requires pre-populated data for good centroid clustering. **HNSW (Hierarchical Navigable Small World)** builds a multi-layer proximity graph. It requires more memory and takes longer to index, but provides substantially superior query queries-per-second (QPS) and near-perfect recall (98%+). HNSW is recommended for production HR assistants.

Q4: What is Hybrid Search and why is pure vector search dangerous in HR?

Answer: Pure vector search relies on semantic concepts and can fail on specific acronyms, form numbers, or legal codes (e.g., 'Form 16', 'Section 80C', 'LOP-2026'). **Hybrid Search** merges dense vector embeddings (for semantic intent, synonyms, and conversational tone) with sparse lexical search (PostgreSQL tsvector / BM25 for exact keyword matching). Scores are combined using Reciprocal Rank Fusion (RRF) or weighted sums: *Score = 0.7 * VectorSim + 0.3 * BM25Rank*.

Q5: How does Reranking improve answer precision?

Answer: Bi-encoders (used to create vector embeddings) compress entire texts into single vectors independently, causing subtle relationship losses. A **Cross-Encoder (Reranker)** processes the user question and candidate chunk together using full cross-attention. In a 2-stage pipeline, pgvector retrieves the top-15 candidate chunks cheaply, and a reranker (e.g., Cohere Rerank, BGE-Reranker) re-scores them to pick the top 3-4 most relevant chunks, boosting context precision by 15-30%.

Q6: How do you solve multi-turn conversation ambiguity in RAG?

Answer: Follow-up questions often contain pronouns and implied context (e.g., User: 'How many casual leaves do I have?' -> 'Can I carry them forward next year?'). A raw vector search on the second query fails. **Solution: Condense/Rewrite Question Step**. Before vector search, an LLM rewrites the conversation history + latest message into a standalone question: *'Can an employee carry forward unused casual leaves to the next year at Ramel?'*. The standalone query is then embedded and retrieved.

Q7: How do you evaluate and benchmark a RAG pipeline?

Answer: Using the **RAGAS** framework across 4 core metrics:
1) **Faithfulness:** Does the answer only contain facts mentioned in the context? (Measures hallucination rate).
2) **Answer Relevance:** Does the generated response directly answer what the employee asked?
3) **Context Precision:** Are the retrieved chunks ranked accurately, with true ground-truth chunks at the top?
4) **Context Recall:** Did the retrieval step fetch all necessary clauses needed to compose the full answer?

Q8: How do you handle PII and sensitive employee data in HR RAG?

Answer: 1) **Pre-Ingestion Redaction:** Mask PAN numbers, Aadhaar, salary figures, and personal phone numbers using regex or Microsoft Presidio before generating embeddings. 2) **Metadata RBAC:** Store department, role level, and clearance tags in Postgres JSONB metadata; append SQL `WHERE` clauses to vector queries based on JWT session. 3) **Audit Trail:** Record every query, retrieved chunk ID, and user ID in an audit log for compliance.

Q9: What is HyDE (Hypothetical Document Embeddings)?

Answer: When employee questions are very terse (e.g., 'maternity leaves?'), question-to-document vector similarity can be weak. HyDE instructs an LLM to hallucinate a plausible document passage answering the question. That synthetic passage is then embedded. Because document-to-document vector distributions align closer than query-to-document distributions, HyDE yields superior retrieval for vague queries.

Q10: What are the latency benchmarks and how do you optimize them for production?

Answer: A well-tuned RAG pipeline should return responses in **under 1.5 seconds**: Vector generation (100-200ms) + pgvector HNSW search (15-30ms) + LLM TTFT (Time-To-First-Token, 300-500ms). Latency optimizations include: 1) Streaming tokens to frontend via Server-Sent Events (SSE). 2) Semantic caching with Redis (cache prior query embeddings and answers). 3) Asynchronous background ingestion queues (BullMQ/Celery).

Deployment Verification Checklist: *Ensure `vector` extension is active in Postgres, HNSW indexes are built with `vector_cosine_ops`, embedding dimensions strictly match model outputs (1536 for OpenAI, 768 for Gemini), and temperature is set <= 0.2 for factual answers.*